Hopfield 网络的实现与探究 *

李宇豪 †

(中山大学物理学院，广东 广州，510275)

2022 年 12 月 15 日

1 Hopfield 网络的实现

首先定义要存储的模式，网课作业中是使用 13 种存储模式，但是经过尝试发现存储 13 种模式时回忆的效果并不好，因此本文中使用了 5 中存储模式。要存储的模式如图1(a) 所示，这是五个数字的 10×10 的点阵图像。将模式存储到网络中，存储后网络的权重矩阵如图1(c) 所示，可以看到，这是一个对称且对角元素为零的矩阵。

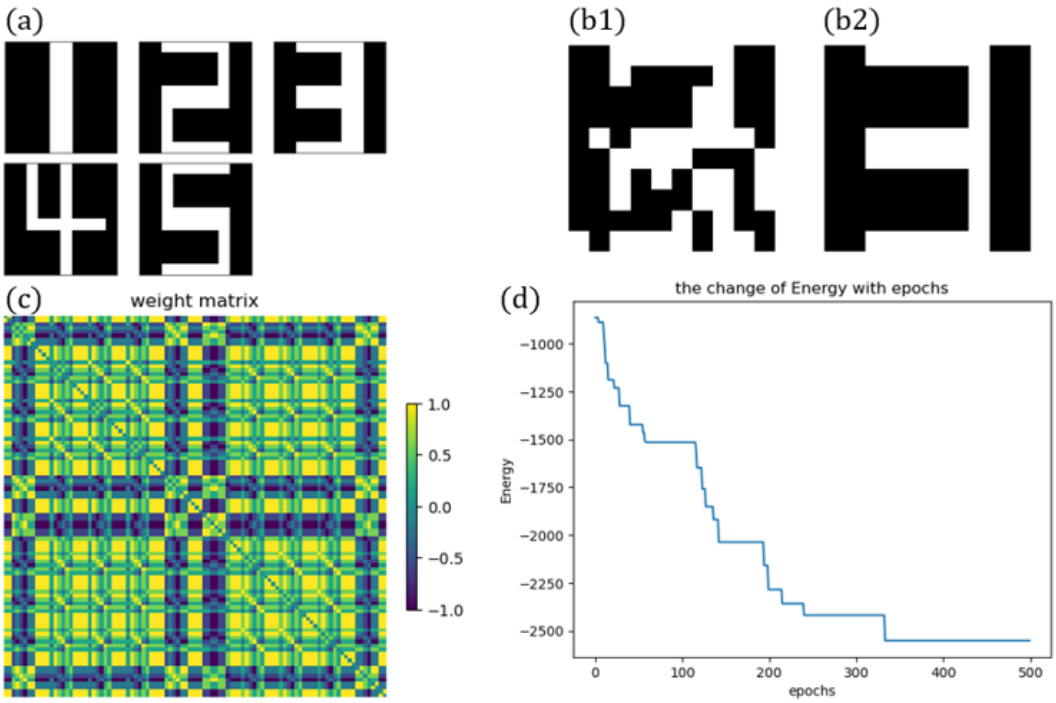

图 1: (a) 网络中存储的模式 (b1) 用于回忆的初始化矩阵 (b2) 回忆结果 (c) 网络的权重矩阵 (d) 训练过程中能量的变化

*SMNN 网课作业

†Email:liyh536@mail2.sysu.edu.cn

我们首先选择一个模式（比如“3”），将原图片随机失真后（如图1(b1)）再放入到网络中进行训练，神经元的更新采用异步方式，即每次随机选择一个神经元进行更新. 经过约 350 次训练后，可以完全回忆起原来的图像，如图1(b2) 所示。整个训练过程中网络的能量在不断降低，如图1(d) 所示，当网络的能量收敛后，说明此时网络的动力学已经演化到一个吸引子中，此后网络的能量和状态不再发生变化。

2 回忆准确率与提示清晰度的关系

为了量化表示提示的清晰度，我们从图片“3”的一百个像素点中随机选择 n 个进行翻转，翻转的像素的比例就可以用来表示提示的清晰程度。如图2 所示，随着翻转的元素比例增加，初始化图像越来越不规则。像素反转比例在 30% 以内时，网络可以完全回忆出数字“3”，但是当初始化的图像过于无序时，即使人来做判断也很难辨别，此时网络的回忆结果是三条横线，这是保留了“2”“3”“5”的共同主要特征的结果。

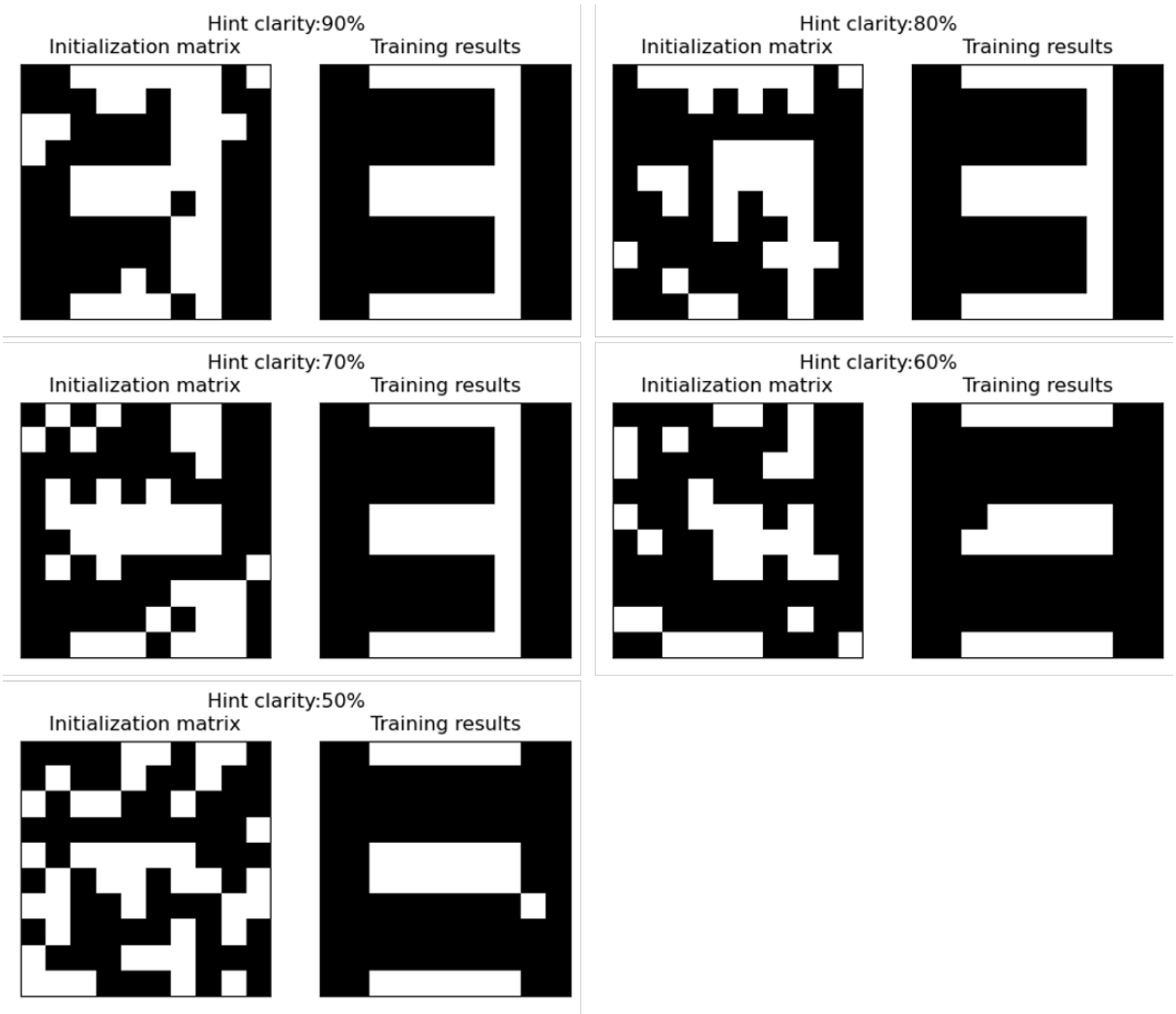

图 2: 不同的提示清晰度下随机失真后的图像与网络的回忆结果

图3 是图2 训练过程中准确率和网络能量的变化，可以看到，提示清晰度较高时，网络可以较快的收敛到准确率为 100%[1]的状态，能量也收敛到一个最低值。提示的清晰度较低时，准确率和能量会收敛到一个次稳定状态，这个状态也对应了网络中的一个吸引子，但是这个吸引子并不是网络最初存储的模式。这一点会在下一小节做一个直观的展示。

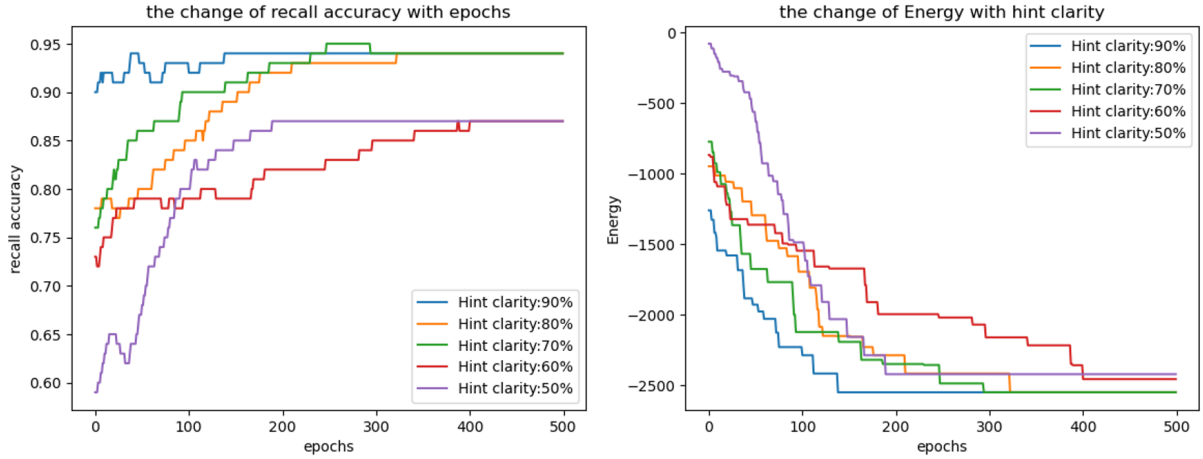

图 3: 不同的提示清晰度下训练时的准确率和能量变化

3 能量景观

由于 100 个神经元的网络可能存在的模式有 2^{100} 种，绘制能量景观难度太大，本节使用一个由 9 个神经元构成的 Hopfield 网络，存储 3 种模式作为例子。

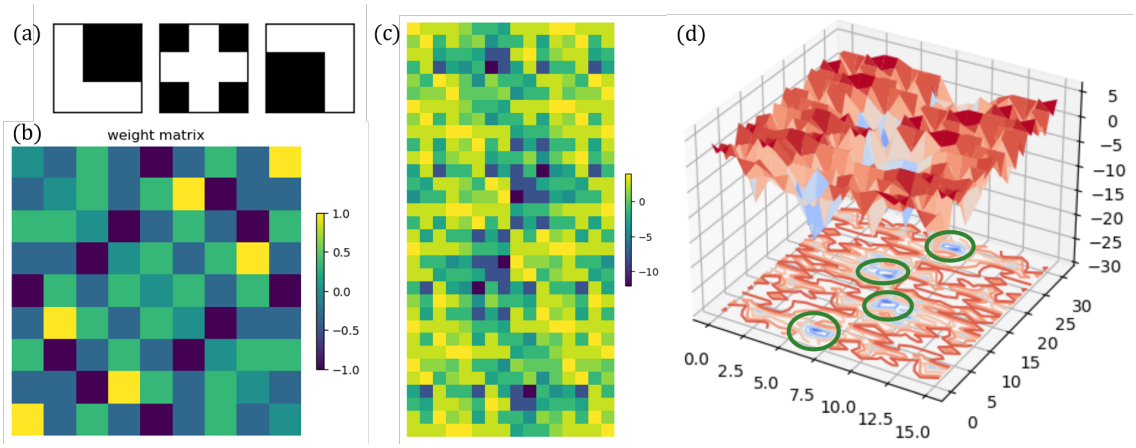

图 4: (a) 存储的三种模式 (b) 9 个神经元构成的 Hopfield 网络的权重矩阵 (c) 能量景观的矩阵可视化 (d) 能量景观的 3D 可视化，xOy 平面上是能量景观的等高线图

[1]由于代码和作图的原因，图3 左图中收敛到的稳定值不是 100%，但是我在代码中没有找到出错的地方。实际上图3 左图最后收敛到的稳定值是两张图片完全相同的状态。

如图4 所示，图4(a) 是该网络存储的三种模式，训练后的权重矩阵如图4(b) 所示，该网络共有 9 个神经元，每个神经元有 ±1 两种取值，因此网络的状态共有 512 种。由于我还没有掌握一些降维的方法，只是将 9 维空间中的状态简单的排列在一起拼成二维和三维数据，如4(c)(d) 所示，这种降维过程损失了高维空间的拓扑信息，与高维空间中的实际形状并不对应。但是仍能看到，能量景观中存在 4 个极小区域（4(d) 中绿色线在等高线图中标出）即网络的四个主要的吸引子。

注意到我们实际上只给网络存储了 3 种模式，因此这四个主要吸引子中包含了次稳定状态，即不是由网络存储的模式产生的，而是多种模式交杂产生的。这种吸引子就会使网络在回忆时收敛到图3 中次稳定的状态。

4 去学习机制

使用去学习机制对 Sec.1 中的网络进行训练，即类似于人脑“做梦”的过程。在使用去学习机制前后分别进行一次 Sec.2 中的训练，结果如图5 所示。可以看到，在经过去学习机制后，五种提示清晰度下都可以较快地收敛到完整准确回忆起原图片[2]。可见这个去学习机制确实可以有效消除次稳定状态，提高回忆的准确率。

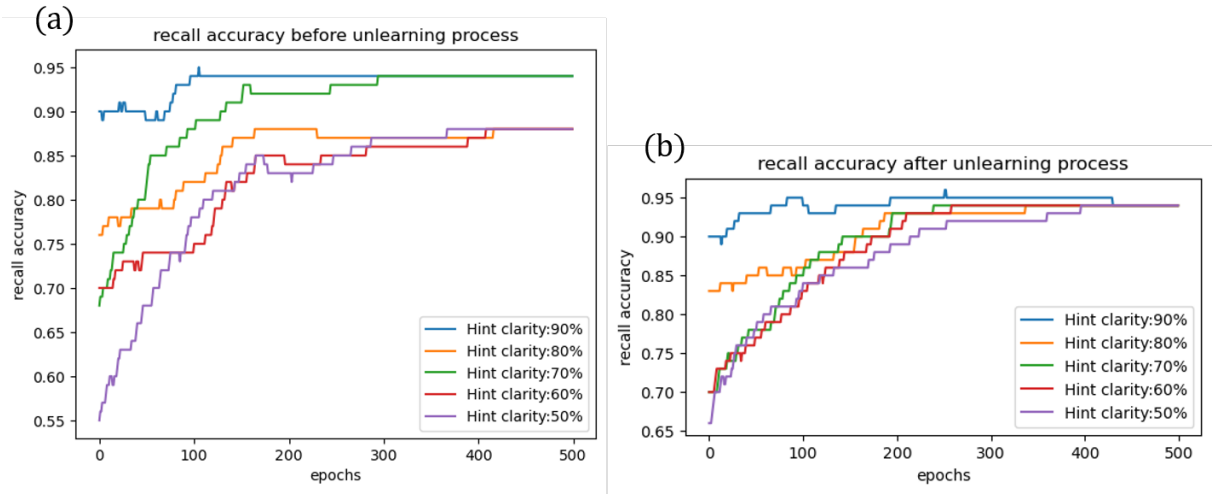

图 5: (a) 使用去学习机制前的训练准确率 (b) 使用去学习机制后的训练准确率

为了直观展示，下面仍用 9 个神经元组织的网络作为实例绘制去学习过程中能量景观的变化。

由于这个网络规模较小，去学习机制的作用效果并不明显。但是仍可以看出，相比图4(d)，图6(a) 在经过一次去学习机制后变得更为平滑（但是没有有效消除次稳定点）。随着去学习次数的增加，能量景观更加平滑，但是出现了很多能量较小的小吸引子。这有点像我在宿舍睡觉睡太多了后醒来神志不清什么都不记得了。

[2]图中的准确率仍是代码绘制部分有误

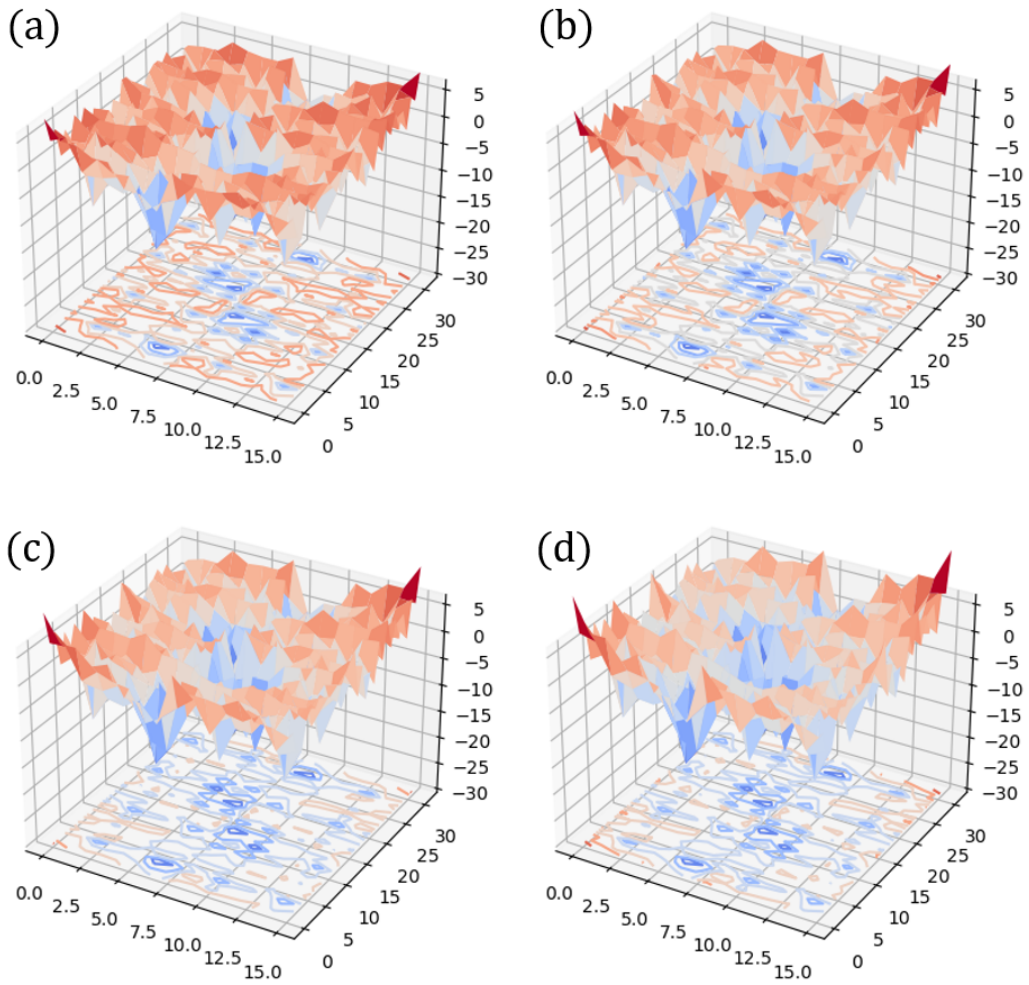

图 6: (a) 去学习机制进行 1 次训练 (b) 去学习机制进行 2 次训练 (c) 去学习机制进行 3 次训练 (d) 去学习机制进行 4 次训练

附录：关键部分代码

```
import matplotlib.pyplot as plt
import numpy as np
import math

# 要记忆的模式数
nb_patterns = 5
pattern_width = 10
pattern_height = 10

# 定义模式（0~9数字的点阵图像，具体数据省略）
patterns = np.array([pattern_1,pattern_2,pattern_3,pattern_4,pattern_5],
                    dtype=float)
```

```
# 存储模式
# 定义权重矩阵
W = np.zeros((pattern_width * pattern_height, pattern_width *
                                        pattern_height))

for i in range(pattern_width * pattern_height):
    for j in range(pattern_width * pattern_height):
        if i == j or W[i, j] != 0.0:
            continue

        w = 0.0

        for n in range(nb_patterns):
            w += patterns[n, i] * patterns[n, j]

        W[i, j] = w / patterns.shape[0]
        W[j, i] = W[i, j]

# 进行回忆
S = np.array(pattern_3)
n_update = 500
hint_clarity = [10, 20, 30, 40, 50]
energy_list = []
recall_acc = []

for n in range(5):
    fig = plt.figure(figsize=(6, 3))

    # 设置随机失真矩阵
    for i in range(hint_clarity[n]):
        j = np.random.randint(0, 10)
        k = np.random.randint(0, 10)
        S[10*j+k] = -S[10*j+k]

    # 绘制随机失真后的图片
    ax = fig.add_subplot(1, 2, 1)
    ax.matshow(S.reshape((pattern_height, pattern_width)), cmap='gray')
    ax.set_xticks([])
    ax.set_yticks([])
    ax.set_title("Initialization matrix")

    # 进行回忆训练，神经元采用异步更新
```

```
    energies = []
    accs = []
    for neuron in range(n_update):
        #print("第", neuron+1, "次训练")
        rand_index = np.random.randint(0, pattern_width*pattern_height)
        #print("选择的神经元编号为: ", rand_index)
        index_activation = np.dot(W[rand_index, :], S)
        #print("该神经元的状态为", index_activation)
        if index_activation < 0:
            S[rand_index] = -1
        else:
            S[rand_index] = 1

        # 计算能量
        energy = -0.5 * np.dot(np.dot(S.T, W), S)
        energies.append(energy)

        # 计算回忆准确率
        acc = 1 - sum([abs(x-y) for (x,y) in zip(S,pattern_3)])/200
        accs.append(acc)

    energy_list.append(energies)
    recall_acc.append(accs)


# unlearning
random_state = np.random.randint(0,2,(100,))
random_state[random_state==0]=-1
plt.matshow(random_state.reshape((pattern_height, pattern_width)), cmap='
                                        gray')
plt.title("the random state")
plt.xticks([])
plt.yticks([])
plt.axis('off')

for i in range(1):
    for neuron in range(100):
        rand_index = np.random.randint(0, pattern_width*pattern_height)
        index_activation = np.dot(W[rand_index, :], random_state)
        if index_activation < 0:
            random_state[rand_index] = -1
        else:
            random_state[rand_index] = 1
```

```
    plt.matshow(random_state.reshape((pattern_height, pattern_width)), cmap
                                        ='gray')
    plt.title("recall result")

    W = W - 0.002 * np.dot(random_state.T, random_state)
    ax = plt.matshow(W)
    plt.colorbar(ax.colorbar, fraction=0.025)
    plt.title("weight matrix  unlearning times="+str(i+1))
```